

Intro to Typst

Yik Jin

Tuesday, 06 October 2026, 1900h

NUS Hackers

yikjin@nushackers.org

1. What is Typst?	2
2. Basics of Typst markup	10
3. Modes	19
4. Code mode	21
5. Math mode	30

1. What is Typst?

1.1. Typst summary

Typst is “an open-source typesetting system and corresponding markup language”

- Helps you make **clean, professional documents** from simple **markup code** (similar philosophy to, but much more powerful than Markdown)
- Easier than using L^AT_EX
- Is a more modern alternative to L^AT_EX

1.2. What Typst can create

Typst can output to various common file types:

- PDF → **today's focus**
 - Cheat sheets
 - Assignments
 - Resume / CV
 - Slides
 - ...
- PNG
- SVG
- HTML (*experimental*)
- Bundle of various files (*experimental*)

1.3. Some examples

Some examples of documents created using Typst:

- [Cheatsheet](#)
- [Datasheet](#)
- [Slides](#)
- ...

1.4. Why not L^AT_EX?

While L^AT_EX is the de-facto industry standard for creating scientific documents, it is unwieldy:

- A mistake in the L^AT_EX markup often gives unhelpful/mysterious error messages (*e.g.* ! Missing \$ inserted. when there is blank lines in math environments)
- Modifying the design and layout of files is tricky
- Compiling L^AT_EX files takes (tens of) seconds
- Not as easy to pick up compared to Typst

1.5. Disambiguation of Typst

The term “Typst” has different meanings in different contexts

When someone refers to Typst, they may mean:

1. The Typst **markup language**
2. The Typst **compiler** (typst binary that compiles the markup language)
3. The Typst **web app** (at typst.app/play)

1.6. What we'll be using today

We'll be writing the Typst markup language today, which needs the typst compiler to generate output files like PDF

For ease-of-use and simplicity, we will be using the **Typst web app** at typst.app/play instead

1.6.1. Typst web app

The Typst web app allows **live-compiling Typst markup** to preview the expected output quickly

- Signing up for an account for today is *optional*

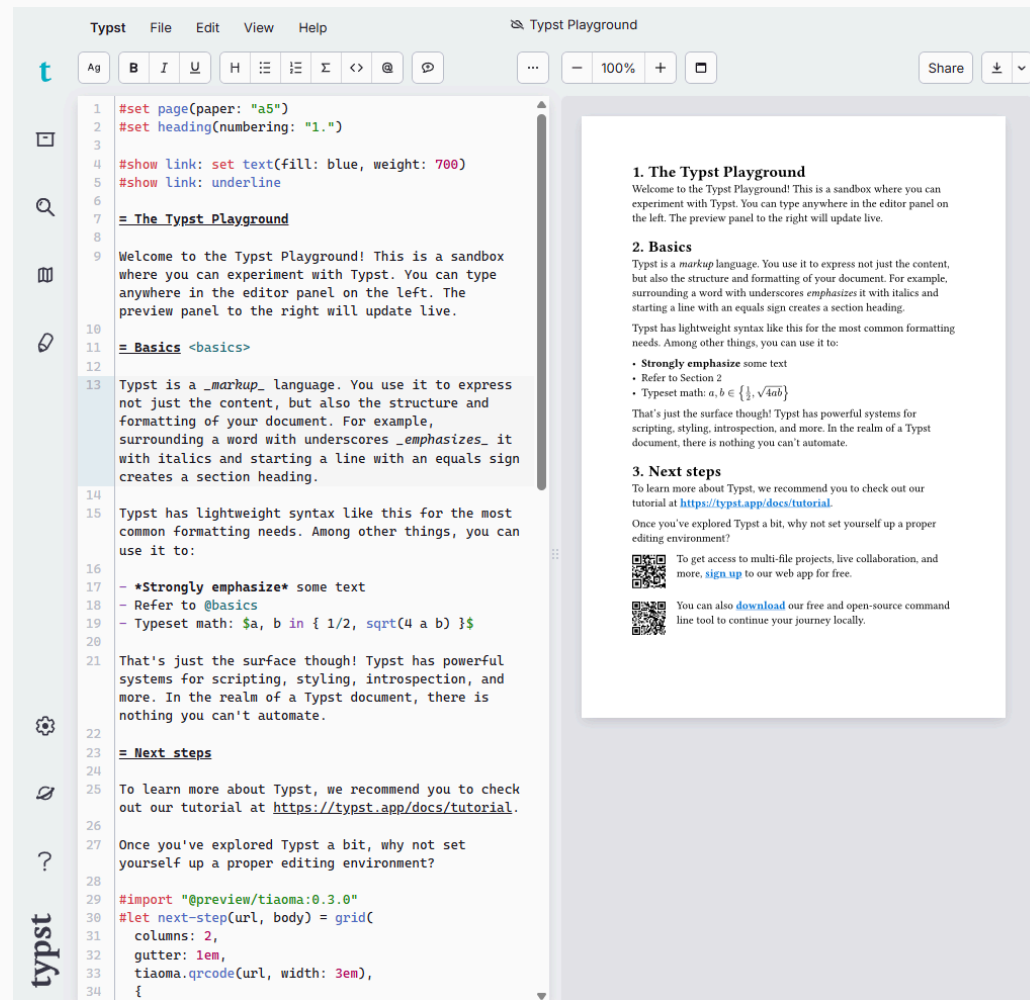


Figure 1: Typst web app

2. Basics of Typst markup

2. Basics of Typst markup

Typst has a simple markup syntax, and is easy to pick up

It has a similar philosophy to Markdown, but Typst syntax is different

2. Basics of Typst markup

Write a line of plain text as-is

A paragraph is separated from another with an
empty line in between

This is a paragraph. It can
have multiple lines.

This is another paragraph.

This is a paragraph. It can have
multiple lines.

This is another paragraph.

2.1. **Bold** and *italics*

Bold words with ***bolded words***

Italicise words with *_italicised words_*

This is a paragraph. It can have ***multiple lines***.

This is *_another_* paragraph.

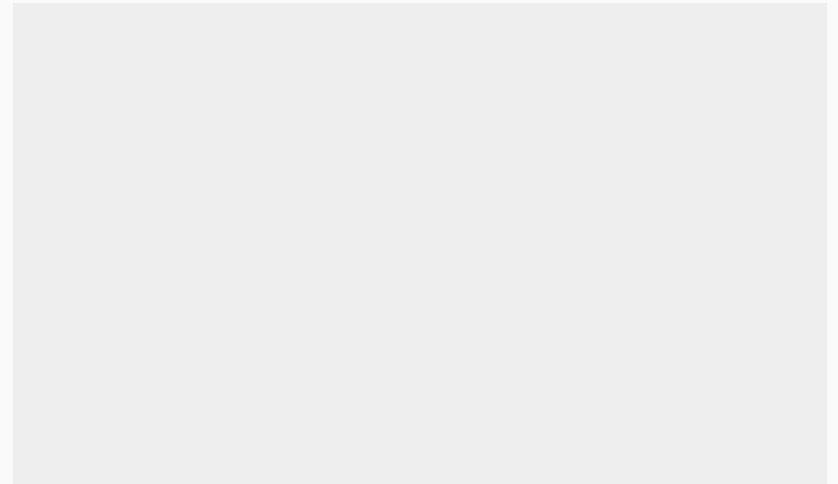
This is a paragraph. It can have **multiple lines**.

This is *another* paragraph.

2.2. Headings

Create a heading with `= Heading 1`

Headings have different levels, denoted by number of `=`s



2.3. Lists

An unordered (*i.e.* bullet point) list starts with -

An ordered (*i.e.* numbered) list starts with +

- Hang out with friends
- Book an appointment

- + Take a straw
- + Use straw to poke lid of bubble tea

- Hang out with friends
 - Book an appointment
1. Take a straw
 2. Use straw to poke lid of bubble tea

2.4. Comments

Add a comment with:

- `//` Comment for single-line comment
- `/*` Comment `*/` for multi-line comment

```
// This is a comment  
// There is no output below
```

```
/* This comment spans  
multiple lines */
```

2.5. Raw text

Add raw/monospaced text with ``text``

This is a ``monospaced text``.

This is a monospaced text.

2.6. Code block

There are 2 ways to display code blocks:

- **Inline** to the paragraph
- **Block level** as a paragraph

Code should be surrounded with triple-backticks, with a Markdown language tag given

```
What is ```rust fn main()```  
in Rust?
```

```
```rust  
fn main() {
 println!("Hello!");
}
```
```

```
What is fn main() in Rust?
```

```
fn main() {  
    println!("Hello!");  
}
```

3. Modes



3. Modes

Typst has 3 different modes:

- **Markup** mode
- **Math** mode
- **Code** mode

3. Modes

Typst has 3 different modes:

- **Markup** mode → what we have been doing thus far
- **Math** mode
- **Code** mode

We have been writing in **markup** mode thus far, allowing us to do formatting of text easily

See a more comprehensive syntax list of markup mode on the [Typst documentation here](#)

4. Code mode



4. Code mode

We'll now use some functions in code mode to insert more objects to our document

Enter code mode with `#`, then you can invoke functions, set variables, *etc.*

4.1. Adding images

Add local or remote images with `image()`, passing in the URI

```
#image("res/nus_hackers.svg")
```



4.1. Adding images

Functions can accept arguments. The `image()` function accepts a named argument “width”.

```
#image("res/nus_hackers.svg",  
width: 50%)
```



4.2. Adding tables

Add tables with `table()`, passing in various arguments as needed

```
#table(  
  columns: 2,  
  table.header(  
    [ *A* ], [ *B* ]  
  ),  
  [ 1 ], [ 2 ],  
  [ 3 ], [ 4 ]  
)
```

| A | B |
|---|---|
| 1 | 2 |
| 3 | 4 |

4.3. Set and show rules

Use `#set` to set properties to all occurrences of some kind of content

- e.g. `#set page(paper: "a6")`

4.3. Set and show rules

Use `#show` to redefine how certain elements are displayed

```
#show "Hackers": name => box[
  #box(image(
    "res/
nus_hackers.svg",
    height: 0.7em
  )) #name]
```

Welcome to Hackers!

Welcome to  Hackers!

4.3. Set and show rules

Use a show-set rule to apply to elements matching the selector

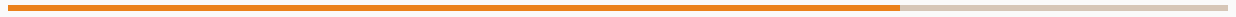
- e.g. `#show heading: set align(center)`

4.4. Setting up the page

For general page customisations, you can use set rules on:

- `text`
- `page`
- `par`
- `heading`
- `document`

5. Math mode



5. Math mode

Let's start typing some math equations now!

Enter math mode with $\$, and type math equations before the closing $\$$$

A linear graph has the form $y = m x + c$. Therefore:

$\$$

$$y = m x + c \backslash$$

$$y - c = m x \backslash$$

$$x = (y - c) / m$$

$\$$

A linear graph has the form $y = mx + c$. Therefore:

$$y = mx + c$$

$$y - c = mx$$

$$x = \frac{y - c}{m}$$

5.1. Inline and block-level math

There are 2 ways to display math:

- `$...$` shows the math **inline** to the paragraph
- `$ ... $` with at least a starting and ending space shows the math in its **own block**

A linear graph has the form
`$y = m x + c$`. Therefore:

`$`

$$\begin{aligned}y &= m x + c \backslash \\y - c &= m x \backslash \\x &= (y - c) / m\end{aligned}$$

`$`

A linear graph has the form $y = mx + c$. Therefore:

$$y = mx + c$$

$$y - c = mx$$

$$x = \frac{y - c}{m}$$

5.2. Variables

Variables must be a character (e.g. m).

Multi-character variables (or just strings) can be displayed between quotes (e.g. `"gradient"`).

A linear graph has the form $y = m x + c$. Therefore:

```
$  
    y = m x + c_"int" \  
    y - c_"int" = m x \  
    x = (y - c_"int") / m  
$
```

A linear graph has the form $y = mx + c$. Therefore:

$$y = mx + c_{\text{int}}$$

$$y - c_{\text{int}} = mx$$

$$x = \frac{y - c_{\text{int}}}{m}$$

5.3. Sub- and super-script

Create sub-scripts with `_`, and super-scripts with `^`

You can group multiple terms with brackets (e.g. $2^{(n+1)}$)

A linear graph has the form $y = m x + c$. Therefore:

```
$  
    y_1 = m x_1 + c_int \  
    y_1 - c_int = m x_1 \  
    x_1 = (y_1 - c_int) / m  
$
```

A linear graph has the form $y = mx + c$. Therefore:

$$\begin{aligned}y_1 &= mx_1 + c_{\text{int}} \\y_1 - c_{\text{int}} &= mx_1 \\x_1 &= \frac{y_1 - c_{\text{int}}}{m}\end{aligned}$$

5.4. Fractions

Create fractions with / (e.g. a/b)

You can group multiple terms with brackets (e.g. (n+1)/(n^2))

A linear graph has the form $y = m x + c$. Therefore:

```
$  
    y_1 = m x_1 + c_int \  
    y_1 - c_int = m x_1 \  
    x_1 = (y_1 - c_int) / m  
$
```

A linear graph has the form $y = mx + c$. Therefore:

$$y_1 = mx_1 + c_{\text{int}}$$

$$y_1 - c_{\text{int}} = mx_1$$

$$x_1 = \frac{y_1 - c_{\text{int}}}{m}$$

5.5. Symbols

Math symbols can be inserted inside math mode

- See [this Typst documentation link](#) for a full list of symbols

```
$  
    m = (dif y)/(dif x) \  
        integral_a^b x^n dif x =  
    [(x^(n+1))/(n+1)]_a^b  
$
```

$$m = \frac{dy}{dx}$$
$$\int_a^b x^n dx = \left[\frac{x^{n+1}}{n+1} \right]_a^b$$

5.6. Functions

You can have function calls in math mode (e.g. `mat()` for matrix, `accent()` for accents over variables)

- See [this Typst documentation link](#) for a list of built-in math functions

```
$  
c = sqrt(a^2 + b^2) \  
m = mat(1, 2; 3, 4)  
$
```

$$c = \sqrt{a^2 + b^2}$$
$$m = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

5.7. Alignment

Math blocks can have alignment markers as well, useful for aligning multiple equations in a block

- `&` is the alignment marker used, alternating alignment from right to left back to right...

```
$  
    y & = m x + c \  
    & = 12  
$
```

$$y = mx + c$$
$$= 12$$

5.7. Alignment

Math blocks can have alignment markers as well, useful for aligning multiple equations in a block

- $\&$ is the alignment marker used, alternating alignment from right to left back to right...

```
$  
a & = 1 && != 2.01234 \  
b & = 3.14159 && != 3  
$
```

$$\begin{aligned} a &= 1 && \neq 2.01234 \\ b &= 3.14159 && \neq 3 \end{aligned}$$

5.8. Exercise

Try typing these math equations in one math block using Typst:

$$c_k = f_p \int_{t_0}^{t_0 + T_p} x_p(t) e^{-j2\pi k f_p t} dt, \quad \forall k \in \mathbb{Z}$$

Phase spectrum $\angle X(f) = \tan^{-1} \left(\frac{\Im[X(f)]}{\Re[X(f)]} \right)$

$$[x_n(t) = \mathcal{F}^{-1}\{X_n(f)\}] \Leftrightarrow [X_n(f) = \mathcal{F}\{x_n(t)\}]$$

5.8.1. Solution

The solution for the previous slide:

\$

```
c_k &= f_p integral_(t_0)^(t_0 + T_p) x_(p)(t) space.thin e^(-j 2 pi k f_p  
t) dif t, space forall k in ZZ \
```

```
\
```

```
"Phase spectrum" angle X(f) &= tan^(-1)(Im[X(f)] / Re[X(f)]) \
```

```
[x_(n)(t) = cal(F)^(-1){ X_(n)(f) }] & arrows.rl [X_(n)(f) = cal(F){  
x_(n)(t) }]
```

\$

Thanks for coming :D